SOFTWARE REQUIREMENT SPECIFICATION (SRS)

Home Loan Management System

---

1. Introduction

1.1 Purpose

The purpose of this document is to describe the requirements for the **Home Loan Management System**. This system enables bank customers to apply for home loans, check eligibility, manage repayments, and perform loan-related operations such as prepayment and foreclosure.

This document is intended for:

- Developers
- Testers
- Project evaluators

1.2 Scope

The system provides an end-to-end solution for managing home loans including:

- Loan application
- Loan approval based on eligibility
- EMI calculation
- Repayment tracking
- Prepayment and foreclosure
- Exporting repayment details

1.3 Definitions

- **EMI**: Equated Monthly Installment
- **Foreclosure**: Closing loan by paying full outstanding
- **Prepayment**: Partial payment before due date
- **Outstanding Balance**: Remaining loan amount

---

2. Overall Description

2.1 Product Perspective

This is a **web-based application** consisting of:

- Frontend: ReactJS
- Backend: Spring Boot
- Database: MySQL

2.2 User Classes

1. Account Holder (Primary User)

- Applies for loan
- Views loan details
- Pays EMI

2. System (Automated)

- Calculates EMI
- Approves/rejects loans
- Generates repayment schedule

2.3 Operating Environment

- Browser: Chrome / Edge
- OS: Windows / Linux
- Backend Server: Spring Boot
- Database: MySQL

2.4 Assumptions & Constraints

- Users are pre-registered
- Fixed interest rate = 7%
- Salary proof is assumed valid
- System simulates bank operations (no real transactions)

---

3. Functional Requirements

3.1 Authentication

Description:

User logs in using credentials.

Input:

- Username
- Password

Output:

- Login success / failure

3.2 Loan Application

Description:

User applies for a home loan.

Inputs:

- Property details
- Loan amount
- Tenure (5–20 years)
- Monthly salary
- Property document

Processing:

- Calculate eligibility

Output:

- Application submitted

3.3 Loan Eligibility Check

Logic:

Eligible Amount = Salary × 50

Conditions:

- If requested ≤ eligible → Approved
- Else:
    - Accept eligible amount OR
    - Cancel application

3.4 EMI Calculation

Formula:

EMI = (P × R × (1+R)^N) / ((1+R)^N − 1)

Where:

- P = Loan amount
- R = Monthly interest rate
- N = Tenure in months

3.5 Loan Approval

Description:

System approves loan based on eligibility.

Output:

- Loan account created
- EMI generated
- Repayment schedule created

3.6 Repayment Schedule

Description:

System generates monthly breakdown.

Each record includes:

- EMI

- Principal
- Interest
- Outstanding balance
- Status

3.7 EMI Payment

Description:

User pays monthly EMI.

Processing:

- Deduct EMI
- Update schedule status = PAID
- Reduce outstanding

3.8 Prepayment

Conditions:

- Minimum 3 × EMI

Processing:

- Reduce outstanding
- Recalculate EMI
- Update future schedule

3.9 Foreclosure

Conditions:

- Allowed after 3 EMIs

Processing:

- Pay full outstanding
- Mark remaining schedule as CANCELLED
- Close loan

3.10 Export

Description:

User downloads repayment schedule.

Output:

- CSV file

3.11 Notifications

System sends:

- Loan approval message
- EMI payment confirmation

---

4. Use Case Diagram

Use Case 1: Login

Actor: User
Flow:

1. User enters credentials
2. System validates
3. Access granted

Use Case 2: Apply Loan

Actor: User

Flow:

1. User enters details
2. System calculates eligibility
3. Application submitted

Use Case 3: Approve Loan

Actor: System

Flow:

1. Check eligibility
2. Approve or reject
3. Calculate EMI
4. Generate repayment schedule

Use Case 4: Pay EMI

Actor: User

Flow:

1. User clicks Pay EMI
2. System deducts amount
3. Updates schedule

Use Case 5: Prepayment

Actor: User

Flow:

1. User enters prepayment amount
2. System validates
3. Updates loan & schedule

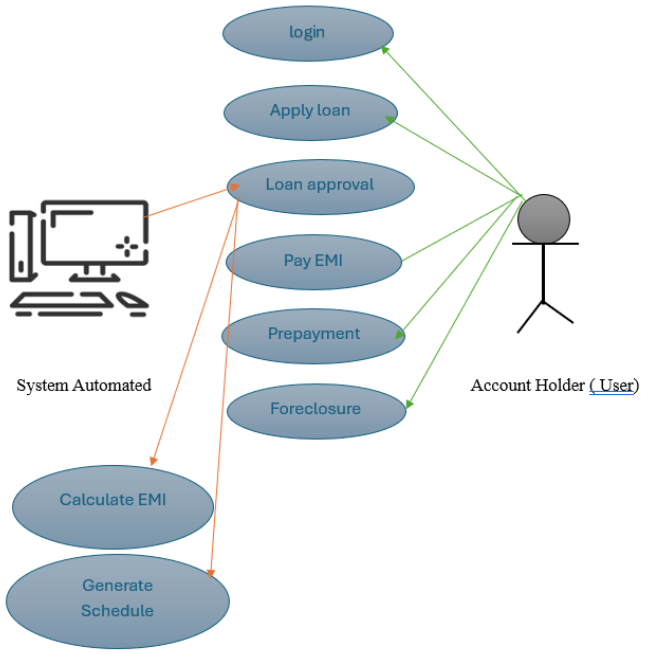
Use Case 6: Foreclosure

Actor: User

Flow:

1. User requests closure
2. System checks eligibility
3. Loan closed

Home loan management use case diagram



5. Non-Functional Requirements

5.1 Performance

- System should respond within 2 seconds

5.2 Security

- Authentication required
- Passwords encrypted

5.3 Usability

- Simple UI for beginners
- Clear form validations

5.4 Reliability

- Data consistency must be maintained
- No data loss

5.5 Scalability

- System should handle multiple users

---

6. Database Requirements

6.1 Database Overview

The system uses a MySQL database to store user, loan, and repayment details.

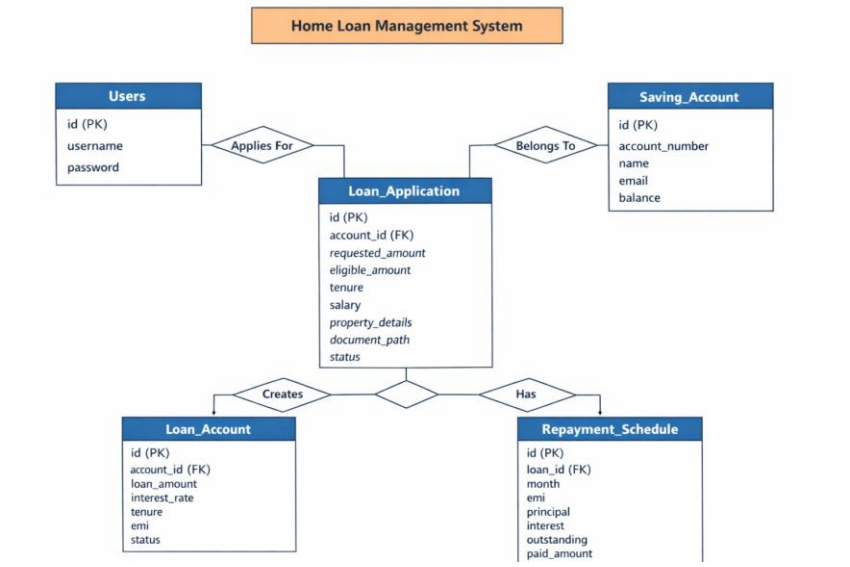
Main tables:

- users
- saving_account
- loan_application
- loan_account
- repayment_schedule

These tables are connected using primary and foreign keys.

6.2 ER Diagram

The ER Diagram shows how different tables are related in the system.



The ER Diagram shows how different tables are related in the system.

Simple Explanation:

- A user has a saving account
- A saving account can apply for loans
- Approved loan becomes a loan account
- Each loan has multiple repayment records

6.3 Table Description

- **users**

Stores login details
Fields: id, username, password

- **saving_account**

Stores user account info
Fields: id, account_number, name, email, balance

- **loan_application**

Stores loan requests
Fields: id, account_id, requested_amount, eligible_amount, tenure, salary, status

---

- **loan_account**

Stores approved loan details
Fields: id, account_id, loan_amount, interest_rate, tenure, emi, status

---

- **repayment_schedule**

Stores EMI details
Fields: id, loan_id, month, emi, principal, interest, outstanding, status

---

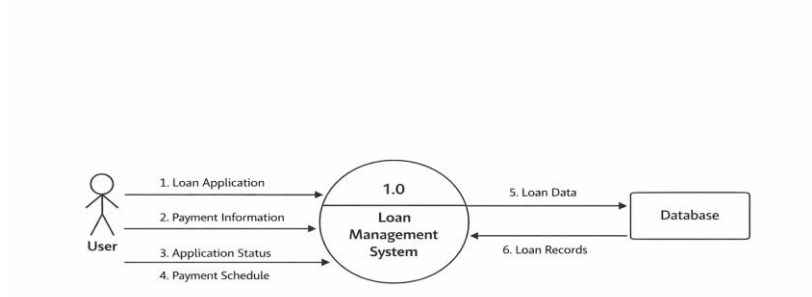
7. System Architecture

Frontend (React)
↓
Backend (Spring Boot)
↓
Database (MySQL)

---

8. Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) represents how data moves within the Home Loan Management System. It shows the interaction between the user, system processes, and the database.



---

9. Future Enhancements

- Real bank API integration
- Mobile application
- Advanced analytics dashboard
- Multiple loan types

---

10. Conclusion

This system provides a complete solution for managing home loans with features like loan application, EMI calculation, repayment tracking, and advanced operations like prepayment and foreclosure. It is designed to simulate real-world banking functionality in a simple and efficient manner.